HW5 Report

R13922213 梁正

1. Implementation

- Parallelism Strategy: I use two-level parallelism. At the kernel level, each GPU thread computes acceleration for one body using a tiled algorithm with shared memory to reduce global memory bandwidth. At the problem level, I run Problem 1 on GPU 0 and Problem 3 on GPU 1 concurrently using separate C++ threads, since these problems have independent initial conditions and outputs.
- Optimization Techniques: I employ double-buffering (ping-pong) for position arrays to avoid synchronization barriers between steps. I precompute device masses for all timesteps into a lookup table with step-major layout for coalesced access. The tiled shared memory approach loads body data cooperatively, reducing global memory traffic. I use the fast rsqrt intrinsic for distance calculations. For Problem 3's branch simulations, I only check for collision every 5000 steps to reduce host-device synchronization overhead.
- 2-GPU Resource Management: Each GPU is assigned via hipSetDevice() before any allocation or kernel launch. Each GPU maintains its own complete set of device buffers, requiring no inter-GPU communication. The main thread spawns two std::thread instances that execute independently and joins them before writing results.

2. Scaling to 4 GPUs

I would keep GPU 0 for Problem 1, then distribute Problem 3's branch simulations across GPUs 1-3. Since testing each gravity device is independent, I would assign different device candidates to different GPUs. For example, with 9 devices, GPU 1 handles devices 0-2, GPU 2 handles 3-5, and GPU 3 handles 6-8. Each GPU runs the main timeline and branches when its assigned devices become reachable. Results are gathered on the host to select the minimum-cost solution. This provides near-linear speedup since branch simulations are embarrassingly parallel.

3. Simulating 5 Gravity Devices

No, it is not necessary. All branch simulations share the same trajectory until a specific device is destroyed. A device can only be destroyed when the missile reaches it, so I run a single main simulation forward and only spawn a branch when the missile reaches each device. Each branch only covers timesteps from

destruction onward, not the full 200,000 steps. Additionally, if destroying an earlier device prevents the collision, later devices may not need full simulation. This provides substantial computational savings compared to 5 independent full simulations.